IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 8 May 2026, accepted 18 May 2026, date of publication 22 May 2026, date of current version 28 May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3696096

RESEARCH ARTICLE

Benchmarking Reference-Free LLM Agent Robustness Under Schema, Policy, and Toolset Drift

MOHAMMAD HASBI ASSIDIQI[1], DANIYAL ALGHAZZAWI[1], (Senior Member, IEEE), SUAAD ALARIFI[1], (Member, IEEE), AND LI CHENG[2]

[1]Department of Information Systems, King Abdulaziz University, Jeddah 21589, Saudi Arabia
[2]Xinjiang Technical Institute of Physics and Chemistry, Chinese Academy of Sciences, Ürümqi 830011, China

Corresponding author: Mohammad Hasbi Assidiqi (massidiqi@stu.kau.edu.sa)

This work was supported by the King Abdulaziz University Endowment (WAQF) and the Deanship of Scientific Research (DSR), King Abdulaziz University, Jeddah, Saudi Arabia.

ABSTRACT Tool-using large language model (LLM) agents fail when external interfaces evolve, yet most evaluations emphasize static competence rather than adaptation under drift. We studied this problem with controlled schema, policy, and toolset perturbations derived from tau2-bench retail tasks and distinguished between a reference-guided replay benchmark and reference-free adaptation. In a reference-guided exact-trajectory setting, GPT-4o mini reached 0.708 task success over 216 trials, establishing that the benchmark contains recoverable structure. We then evaluated five OpenRouter-served models—GPT-4o mini, GPT-4o, Kimi K2, DeepSeek V3, and Qwen3-32B—under a reference-free protocol scoring end-state task completion across 270 trials per model-condition cell (30 scenarios $\times$ 3 episodes $\times$ 3 drift families). Static baselines range from 0.130 to 0.432 across models. Blind replay consistently degrades reference-free task success relative to static for four of five models; Kimi K2 is the exception, for which both blind replay and rules-only significantly improve over the static baseline. Rules-only shows model-dependent effects, significantly improving for Kimi K2 and Qwen3-32B while showing no significant difference for the remaining three models. A normalization-ablation condition confirms that the drift-inversion layer is load-bearing for toolset drift; schema-obedience determines whether this layer is effective. Within the three evaluated drift types and controller conditions, the reference-guided replay advantage does not transfer to reference-free task success, and the primary barriers are hidden-state reconstruction and branch selection across multiple candidate orders and products.

INDEX TERMS Continual learning, drift adaptation, large language models, benchmarking, tool use.

I. INTRODUCTION

Large language model (LLM) systems are increasingly deployed as agents that plan, invoke tools, retain useful experience, and improve behavior over time. In that setting, learning is not only a training-stage concept; it is an operational requirement. A useful agent must adapt when tasks recur, when feedback accumulates, and when the environment no longer matches the assumptions baked into its tool definitions. Dedicated surveys on continual learning for LLMs and agentic-AI design consistently treat adaptation as a core operational capability, while noting that most deployed systems remain training-free, relying on fixed prompts and heuristic reflection rather than principled online learning [1], [2], [3], [4].

This broader problem is naturally framed as continual learning for LLM agents. The central question is not only whether a model can solve a task once, but whether an agent can update behavior under non-stationarity without sacrificing reliability or deployment practicality. Replay-centered methods are especially relevant because they offer a lightweight, empirically stable mechanism for preserving

The associate editor coordinating the review of this manuscript and approving it for publication was Maria Chiara Caschera.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

and reusing experience under changing conditions [5], [6], [7], [8], [9]. Yet most existing replay evidence comes from classification-style streams, where actions are simple outputs and success is not mediated by multi-step tool use, interface contracts, or external environment feedback. First trajectory-level replay systems for LLM agents have recently appeared [10], but general, tool-agnostic replay with reliable performance guarantees under interface drift remains an open research frontier.

Tool-using agents expose this gap sharply. In real deployments, tool-facing environments drift through argument-schema changes, policy-anchor requirements, and tool renaming. Empirical studies show that even modest perturbations to tool names, parameter names, and response schemas substantially degrade agent reliability, revealing that current models overfit to surface metadata rather than underlying tool semantics [11], [12]. At the same time, realistic tool-use benchmarks show that end-to-end reliability remains low and that headline capability claims can be distorted by benchmark and API instability [13], [14]. The practical importance of this problem is therefore larger than a narrow tool-calling bug: if LLM agents are to function as learning agents in production, they must adapt to interface drift in ways that are measurable, reproducible, and stronger than static prompting or hand-written repair rules.

This study targets that gap with a controlled protocol designed around drift robustness rather than static benchmark score maximization. We evaluate three research questions. RQ1 asks how much performance a reference-guided replay condition can recover under matched schema, policy, and toolset drift. RQ2 asks whether memory-based controllers still improve outcomes once evaluation-time reference access is removed and success is measured on the upstream environment rather than by exact trajectory imitation. RQ3 asks what failure modes persist across the three evaluated reference-free controller conditions and what the normalization-ablation condition reveals about the drift-inversion layer's contribution to toolset recovery.

Our contributions are threefold. First, we introduce a controlled drift benchmark that isolates three mechanically verifiable drift types and separates a reference-guided upper-bound condition from a reference-free adaptation regime—providing a diagnostic decomposition framework rather than a new algorithm. Second, we provide a reference-free task-success evaluation path on the upstream tau2 retail environment that executes drift-normalized tool calls and scores end-state task completion, enabling reproducible comparison of controller conditions without requiring reference trajectories. Third, we report empirical results from five models across three drift families showing that reference-guided replay performance does not transfer to the reference-free regime under the three evaluated controller conditions, that blind replay consistently degrades performance for four of five models while rules-only shows model-dependent effects, and that hidden-state reconstruction and branch selection are the key barriers for deployable drift adaptation within this evaluation scope.

II. RELATED WORK

Prior work establishes that realistic tool-use reliability remains low when evaluation moves beyond single-turn invocation toward multi-step interactive sessions. WildToolBench reports that no evaluated model exceeds 15% session accuracy under multi-turn, multi-task conditions with implicit user intent and instruction transitions [13]. In adjacent interactive environments, WebArena, OSWorld, and WebVoyager similarly report substantial end-to-end gaps under long-horizon web and operating-system tasks [15], [16], [17]. StableToolBench adds an evaluation-stability perspective by showing that tool availability and API drift can materially change observed performance, motivating controlled and repeatable tool-evaluation infrastructure [14]. Together, these studies motivate treating tool-use performance as a robustness and adaptation problem rather than a static capability score. The three drift operators evaluated here—argument-key renaming (schema drift), tool-name remapping (toolset drift), and prerequisite-step insertion (policy drift)—correspond to the most common breaking-change categories documented in API evolution studies: parameter renaming, endpoint renaming, and new mandatory preconditions. API stability analyses of publicly available interfaces consistently find that breaking changes of these types appear frequently across version histories and that clients without explicit adaptation logic fail silently or with misleading errors [14]. Our controlled-injection protocol differs from empirical API evolution studies in a key respect: rather than observing naturally occurring drift in production systems, we inject precisely specified, bijective drift under a known inversion manifest, enabling causal isolation of the adaptation problem independent of drift-detection overhead. This design makes the evaluation analytically tractable but explicitly scopes our findings to the subproblem of adapting to drift once it has been detected and inverted—a simplification we acknowledge in Section VI.

Each drift operator maps onto a realistic deployment analog. Schema key rename corresponds to versioned REST API field renames—a common breaking change in API evolution where a renamed request parameter invalidates all prior client integrations that do not explicitly handle the new key. Policy step prepend corresponds to newly mandated consent flows or two-factor authentication gates introduced by regulatory or compliance changes, where an agent that was previously effective must now issue an additional confirmation action before any task-relevant call. Tool-name remap corresponds to service consolidation or vendor switching, where an endpoint is migrated to a new provider with a different naming convention while preserving the underlying argument schema and semantics. These real-world analogs establish that the three controlled drift operators are not arbitrary synthetic perturbations but representative instantiations of the most common categories of breaking interface change encountered in production deployments.

General AgentBench further reports a ''verification gap''—difficulty selecting correct trajectories among candidates—that closely parallels the branch-selection

failures observed in our reference-free evaluation; our results concretize this gap at the tool-call and entity-selection level under explicit interface drift [18].

Two evaluation frameworks closely aligned with our end-state scoring approach are Agent-Diff [19] and MCP-Bench [20]. Agent-Diff benchmarks LLM agents on enterprise API tasks (Slack, Box, Linear, Google Calendar) by defining task success as whether the expected state-difference (ΔS) was applied to a sandboxed environment, closely analogous to our binary DB-state match; the key distinction is that Agent-Diff targets closed-world invariants in stable API settings, while our protocol introduces programmatic drift on a fixed environment. MCP-Bench evaluates tool use across 28 live MCP servers spanning 250 tools, stressing cross-tool coordination, schema compliance, and tool disambiguation under fuzzy instructions without explicit tool names; its focus on multi-server orchestration and naming inconsistencies bears directly on our schema and toolset drift families. For the hidden-state reconstruction bottleneck identified in this paper, Yang et al. propose STITCH—structured indexing of trajectory steps with a contextual intent cue comprising the latent goal, action type, and salient entity types, enabling intent-compatible memory retrieval and coreference suppression [21]; a STITCH-like belief store could directly target the order/product disambiguation failures observed in this study. Finally, RDumb++ [22] extends continual test-time adaptation with entropy-based and KL-divergence drift detectors that trigger principled model resets, replacing fixed periodic schedules; integrating such mechanisms would remove the manifest-drift assumption (the requirement that the drift bijection be known in advance) and increase ecological realism in our evaluation protocol.

Continual-learning research provides the main technical basis for replay as an adaptation mechanism. Online Continual Learning with Maximally Interfered Retrieval shows that replay improves retention when memory sampling prioritizes examples most vulnerable to interference [8]. Subsequent work studies replay selection and stability-plasticity trade-offs under different resource and drift regimes, including abrupt representation-change control, adaptive memory replay, dual-branch online continual learning, robust self-adjusting ensembles for drifting streams, and adaptive memory realignment for concept drift [9], [23], [24], [25], [26]. These findings consistently indicate that replay quality, not only replay existence, governs adaptation outcomes. However, most evidence is derived from classification-style continual-learning streams, leaving open whether replay can transfer to tool-calling agents where failure is dominated by structured action errors and interface-level drift.

Recent studies on agentic-system design broaden context but also highlight gaps in explicit drift adaptation. A healthcare agent survey with seven-dimensional taxonomy reports that adaptation-oriented capabilities such as drift detection and mitigation are rarely implemented, despite growing use of planning and tool-using agents [4]. The deployment momentum of agentic systems in consequential real-world settings establishes that agents face these interface-change challenges in practice—yet no prior controlled evaluation isolates the causal contribution of replay under matched schema, policy, and toolset drift. The present study addresses that gap with a replay-centered protocol and explicit static/rules baselines under matched holdout conditions.

Several complementary threads further contextualize the reference-free adaptation problem. Reasoning with Language Model is Planning with World Model (RAP) [27] demonstrates LLM-based planning with explicit world-model rollouts, showing gains within stable interfaces; our results suggest that interface drift can break stored plan templates regardless of planning quality. Gorilla [28] evaluates LLM tool-call faithfulness at scale and finds that hallucinated or misformatted tool calls remain a persistent failure mode even without drift—a baseline difficulty that our drift conditions compound. WebRL [29] closes end-to-end gaps in web agents via self-evolving curriculum reinforcement learning; translating this to our setting would require an online RL signal from the upstream environment evaluator, which is a promising but substantially more resource-intensive direction than the replay-based controllers evaluated here. In continual-learning settings, Aljundi et al. [30] show that gradient-based sample selection—framed as minimizing interference constraints—improves replay quality in online continual learning, reinforcing that replay quality governs adaptation outcomes more than replay quantity. WILDS [31] provides a multi-domain benchmark for distribution shift that shares the ethos of our controlled-drift protocol; our study differs by targeting structured, programmatic interface drift rather than covariate shift in model inputs.

Planning and memory agents provide relevant prior art for the memory-based controllers evaluated here. ReAct [32] interleaves chain-of-thought reasoning with action generation, demonstrating that structured reasoning traces can improve tool-call accuracy in multi-step tasks. Reflexion [33] extends this approach by adding verbal reinforcement over episode transcripts—a form of experience replay analogous to our memory-cue retrieval, but operating over self-generated critique rather than stored action templates. Voyager [34] demonstrates open-ended skill accumulation in embodied environments through a curriculum over stored skill programs, motivating the idea that replay quality and retrieval structure matter for long-horizon agents. The contrast between these planning-memory agents and our reference-free results is instructive: prior gains from replay and reflection typically operate within a fixed interface, whereas our evaluation exposes the additional challenge posed by interface drift that breaks stored action templates at the call-structure level. This contrast motivates treating interface-level drift adaptation as a distinct evaluation problem rather than an extension of in-context memory or reflection.

III. METHODS

We evaluated adaptation under controlled tool drift on a retail-domain task set adapted from tau2-bench [35].

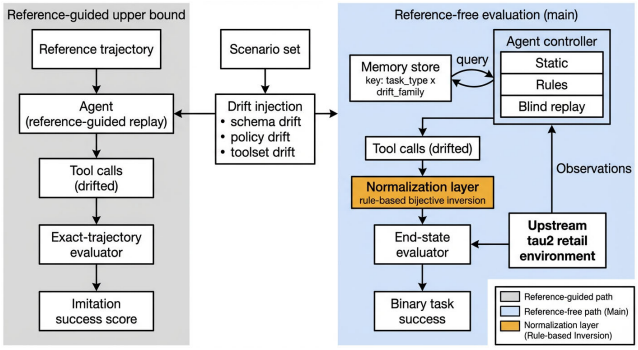


FIGURE 1. Evaluation pipeline showing two incommensurable regimes: the reference-guided path (left, gray) provides a deterministic upper bound via exact-trajectory replay, and the reference-free path (right, blue) scores end-state task completion on the upstream environment without reference access.

The primary reference-free task-success evaluation used five OpenRouter-served models: `openai/gpt-4o-mini` (GPT-4o mini), `openai/gpt-4o` (GPT-4o), `moonshotai/kimi-k2` (Kimi K2), `deepseek/deepseek-chat` (DeepSeek V3), and `qwen/qwen3-32b` (Qwen3-32B) [36]. These five models were selected to represent a range of capability levels and model families accessible via the OpenRouter API within the experiment's compute budget: two OpenAI models spanning that provider's frontier range, and three Chinese frontier models from distinct providers (Moonshot AI, DeepSeek, and Qwen/Alibaba Cloud). This cross-provider selection tests whether the cross-model adaptation pattern holds across diverse model families; the omission of Anthropic frontier models (Claude Opus, Claude Sonnet) is noted as a limitation in Section VI. All model calls used greedy decoding (temperature $=$ 0) with no nucleus sampling; the scenario-shuffle random seed was fixed at 20260310 for reproducibility. No additional system prompt was used beyond the per-scenario task description and tool definitions provided by the tau2 retail environment. We injected three drift families at inference time. *Schema drift* renames one or more required argument keys in the tool schema while preserving semantics (e.g., `order_id` → `order_ref`). *Policy drift* requires that a designated policy-confirmation tool call be issued as the *first* action of any episode; any other tool call issued before this confirmation is rejected by the upstream environment with an error response, effectively blocking progress until the agent issues the confirmation. *Toolset drift* remaps tool names to new identifiers (e.g., `cancel_order` → `terminate_order`) while preserving argument schemas and semantics. Train and eval sets were disjoint by scenario ID while remaining in the same retail-domain pool and using the same drift operators.

Figure 1 provides an overview of both evaluation paths. This study separates two evaluation regimes. The first is a reference-guided upper-bound condition designed as a deterministic upper-bound comparator. In that setting, replay memory stores train-derived signatures and, after retrieval, activates deterministic replay against the scenario reference trajectory; success is measured by exact-trajectory imitation. This regime establishes that the benchmark contains recoverable structure: under reference-guided deterministic replay, GPT-4o mini reached 0.7083 task success over 216 eval trials (95% CI [0.6428, 0.7680], Clopper–Pearson). The 216 eval trials for the reference-guided condition come from a separate reference-guided evaluation run of 24 eval scenarios $\times$ 3 drift families $\times$ 3 episodes, using the exact-trajectory imitation evaluator; this run is independent of the reference-free evaluation (30 eval scenarios $\times$ 3 drift families $\times$ 3 episodes $=$ 270) reported in the remainder of this section. The reference-guided condition is not a deployable reference-free adaptation setting, and the two evaluators are incommensurable—exact-trajectory imitation versus end-state task completion—so no direct subtraction across regimes is warranted.

The second regime is the main evaluation for this paper: reference-free task success on the upstream tau2 retail environment. In this regime, drifted tool names and argument keys are normalized back to the upstream API by a deterministic, rule-based mapping layer. Drift operators applied during injection are bijective: each drifted name or key has exactly one upstream counterpart specified in the drift manifest. The normalization layer inverts this mapping before each tool call is dispatched to the upstream environment, so the environment evaluator always receives calls in the canonical upstream schema. Normalization failures can occur when an agent produces a tool name or key that was not part of the drift manifest (e.g., a hallucinated tool name); in such cases, the call is passed through unchanged and typically produces an upstream error. In the primary reference-free evaluation runs, the normalization failure rate was zero: no agent produced a tool name or argument key outside the drift manifest across any of the up to 270 eval trials per model-condition cell and across all three controller conditions (static, rules-only, and blind replay). This indicates that normalization failures are not a source of noise in the reported success rates under the current drift protocol. These pass-through failures are included in the task-failure count and thus conservatively penalize the agent. The normalization layer itself is not a source of agent capability: it does not repair malformed argument values, does not infer missing required fields, and does not select among multiple plausible branches. Model tool calls are executed against the real environment, and success is scored by the upstream environment evaluator using end-state task completion rather than exact imitation of one gold sequence. The toolset-drift results reported in Section IV should be interpreted within the scope of this bijective-scaffold assumption: the evaluation isolates adaptation difficulty under known-manifest inversion and does not measure the combined problem of autonomous drift detection plus adaptation. The upstream evaluator computes a binary reward via database-state match: after all agent tool calls are executed, the final database state (order records, product inventory, user account fields) is compared against the expected post-task ground-truth state. A trial is scored

as success (reward $=$ 1.0) if and only if all task-relevant database fields match the ground-truth state exactly; partial matches receive reward $=$ 0. No partial credit is awarded, so a correct tool sequence that ends with a single wrong field value (e.g., an incorrect `order_id`) still scores as failure. The evaluation used 10 train scenarios and 30 disjoint eval scenarios, with 3 repeated episodes per condition, yielding up to 270 eval trials per model-condition cell (30 scenarios $\times$ 3 episodes $\times$ 3 drift families). Empirically, the fraction of (scenario, drift-family) cells showing different success/failure outcomes across the three episodes ranges from 5.6% (Kimi K2) to 58.9% (Qwen3-32B) in the static condition, consistent with model-specific LLM API non-determinism at temperature=0; the three episodes therefore contribute genuine outcome variation and $n = 270$ is the reported trial count. Each eval scenario was evaluated under each of the three drift families (schema, policy, toolset), producing a balanced distribution of 90 trials per drift family per condition. Train and eval scenario sets were drawn from disjoint scenario IDs within the same retail-domain pool and used the same drift operators.

Replay memory in the reference-free setting used keyed retrieval over task-pattern signatures and drift type. Each memory entry was indexed by three fields: task type (the top-level retail action, e.g., exchange, return, cancel), drift family (the injected drift operator: schema, policy, or toolset), and source scenario identifier (the train scenario from which the entry was derived). Retrieval used exact matching on task type and drift family; source scenario identifier served as a tiebreaker when multiple entries matched. We did not experiment with dense semantic encodings or embedding-based approximate retrieval; template-signature matching was chosen to isolate the effect of structured drift-type cues from unstructured similarity search.

Controller stack. Three reference-free controller conditions share the same base LLM call and differ only in which repair stages are active at each step. Given a scenario, drift metadata, memory store $\mathcal{M}$, enabled rule set R, and step hints H, each episode step proceeds as follows:

1) *LLM call*: invoke the model with current messages and tools at temperature $= 0$, obtaining predicted action $\hat{a}_t$.
2) *Memory-cue repair* (active in the blind-replay condition when $H[t]$ exists): overwrite or augment $\hat{a}_t$ with the step-t hint retrieved from $\mathcal{M}$.
3) *Rule repair* (active when $R \neq \emptyset$): apply deterministic, drift-type-specific fixes to $\hat{a}_t$. For *schema drift*: remap any drifted argument key to its canonical form. For *policy drift*: prepend the required policy-confirmation call if the agent's first action omits it. For *toolset drift*: rewrite the predicted tool name if it matches a known remapped form.
4) Execute the (possibly repaired) action against the upstream environment; update the state cache from the returned observation.

Static uses step 1 only. **Rules-only** uses steps 1 and 3. **Blind replay** uses steps 1 and 2 (hint at step 0 only), reflecting that retrieval is attempted without access to reference trajectories.

None of these controllers was allowed to access evaluation reference trajectories. More elaborate controller variants—multi-step memory cues, observation-based argument repair, and planning-augmented disambiguation—are discussed in Section V as candidate directions for future evaluation; they were not evaluated in this study and their mechanistic descriptions are omitted from this section. Adaptation in this study refers to pre-episode context augmentation (memory cues and repair rules); within-episode error-driven adaptation—where the agent revises its behaviour in response to environment error messages during execution—is outside the scope of the current evaluation. No updated documentation is provided to the agent during the drift phase: the drifted tool schema is presented as authoritative, and the normalization layer silently inverts drift mappings before dispatch without informing the agent that any interface change has occurred.

The primary estimand in the reference-free regime was mean binary task success on eval trials under the upstream environment reward. Secondary diagnostics included average matched steps and memory-activation counts. We report the reference-guided replay result as motivating evidence for the two-regime evaluation design and treat the reference-free task-success results as the main causal evaluation for adaptation claims.

With $n = 270$ binary trials per model-condition cell and a two-tailed test at $\alpha = 0.05$, the design achieves 80% power to detect a difference in success rates of approximately 0.12 (12 percentage points). The primary finding—blind replay consistently underperforming static—involves observed differences ranging from 0.06 to 0.26 across models, all above this detection threshold. Rules-only effects are smaller and model-dependent (ranging from -0.09 to $+0.08$ relative to static); the design is adequately powered to detect or rule out effects at this scale, though individual between-model comparisons at smaller effect sizes remain underpowered.

The two evaluation regimes serve as complementary analytical lenses rather than competing measurement conditions. The reference-guided condition establishes that recoverable structure exists in the benchmark: when a complete repair trajectory is provided, high task success is achievable—the upper-bound ceiling exists. The reference-free condition establishes whether in-context adaptation strategies can close the gap under operational conditions without reference access. The contrast between the two regimes is a decomposition argument, not a direct score comparison: incommensurable evaluators preclude direct subtraction, but the gap motivates the conclusion that the binding bottleneck is hidden-state reconstruction and branch selection, not execution of a known repair.

IV. RESULTS

RQ1 (reference-guided upper bound). We first report the reference-guided replay result only as an upper bound. Under the reference-guided replay condition, GPT-4o mini reached 0.7083 success over 216 eval trials. This number establishes that the benchmark contains recoverable structure, but it does not by itself show that a reference-free agent can adapt

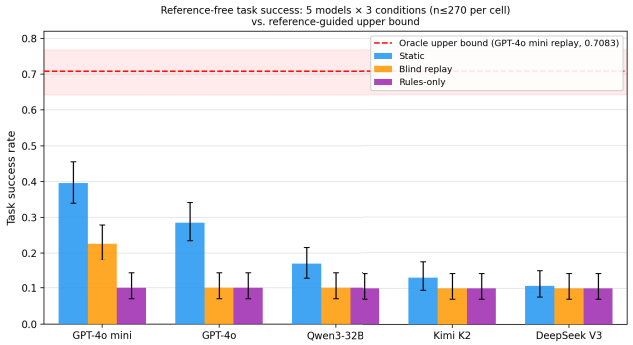


FIGURE 2. Reference-free task-success rates for five models under three controller conditions (static, blind replay, rules-only). The dashed line shows the reference-guided upper bound for GPT-4o mini ($n = 216$); it uses an incommensurable evaluator (exact-trajectory imitation, not end-state task completion) and should be read as a qualitative reference point, not a directly comparable data point. Error bars are 95% Wilson confidence intervals. Blind replay significantly degrades performance below the static baseline for four of five models; rules-only shows model-dependent effects, improving significantly for Kimi K2 and Qwen3-32B. See Table 2 for McNemar's paired tests.

online under drift. The two regimes use incommensurable evaluators—exact-trajectory imitation versus end-state task completion—so no direct subtraction is warranted; the comparison is qualitative.

RQ2 (reference-free adaptation). Table 1 reports task-success rates for all 15 model-condition cells ($n = 270$ per cell, all three drift types combined). Static-condition rates span a wide range: Qwen3-32B reaches 117/270 = 0.432 (95% CI [0.376, 0.493]), GPT-4o mini reaches 107/270 = 0.396 ([0.340, 0.456]), GPT-4o reaches 85/270 = 0.315 ([0.262, 0.373]), DeepSeek V3 reaches 39/270 = 0.144 ([0.108, 0.191]), and Kimi K2 reaches 35/270 = 0.130 ([0.095, 0.175]).

The adaptive controllers yield a differentiated pattern across models, summarised by paired McNemar's tests in Table 2. **Blind replay** significantly degrades task success relative to static for four of five models: GPT-4o mini ($\chi^2 = 28.12$, $p < 0.001$), GPT-4o ($\chi^2 = 29.09$, $p < 0.001$), DeepSeek V3 ($\chi^2 = 7.69$, $p = 0.006$), and Qwen3-32B ($\chi^2 = 33.44$, $p < 0.001$). The exception is Kimi K2, for which blind replay is significantly better than static ($\chi^2 = 10.24, p = 0.001$; 0.193 vs. 0.130). **Rules-only** shows model-dependent effects: it significantly improves over static for Kimi K2 ($\chi^2 = 15.75$, $p < 0.001$; 0.211 vs. 0.130) and Qwen3-32B ($\chi^2 = 7.29, p = 0.007$; 0.526 vs. 0.432); it is not significantly different from static for GPT-4o ($p = 1.000$) or DeepSeek V3 ($p = 1.000$); and it shows a borderline effect for GPT-4o mini ($p = 0.052$; 0.352 vs. 0.396). A logistic regression with model identity and controller condition as fixed effects and drift family as a covariate confirms these directions: blind replay carries $\beta = -0.591$ (OR = 0.554) relative to the static reference; rules-only carries $\beta = +0.133$ (OR = 1.142), a smaller and model-dependent effect. These results are also shown in Figure 2.

Note on statistical precision. Wilson 95% confidence intervals are reported for all reference-free cells (Table 1); the reference-guided upper bound uses Clopper-Pearson. At $n = 270$ the Wilson CI half-width is approximately ± 0.036 for rates near 0.10 and ± 0.060 for rates near 0.40. The paired McNemar's tests (Table 2) show statistically significant differences for blind replay versus static in four of five models and for rules-only versus static in two of five models; individual between-model comparisons at small effect sizes remain underpowered, as noted in Section VI.

RQ3 (failure modes and normalization). The normalization-ablation condition (Table 3; see caption for the data-quality note on "without normalization" rates) isolates the contribution of the drift-inversion layer. GPT-4o mini drops overall from 0.396 to 0.322 when normalization is removed; the reduced $n = 236$ reflects 30 toolset-drift episodes that terminated early due to upstream errors on unmapped tool names—evidence that, without normalization, toolset drift renders the environment unresponsive for schema-obedient models. The other four models produced no early terminations ($n = 270$), indicating that they did not call the drifted tool names at all; the mechanism and its asymmetric implication are discussed in Section V. Because the

TABLE 1. Reference-free task-success results for five models under three controller conditions ($n = 270$ trials per cell; all three drift types combined). Columns show success count k, overall rate, and 95% Wilson confidence interval. The reference-guided upper bound for GPT-4o mini (0.708, $n = 216$, Clopper–Pearson CI [0.643, 0.768]) is reported in the text (RQ1) and shown as a dashed line in Figure 2.

Model	Condition	k	n	Rate	95% Wilson CI
GPT-4o mini	static	107	270	0.396	[0.340, 0.456]
	blind replay	61	270	0.226	[0.180, 0.280]
	rules-only	95	270	0.352	[0.297, 0.411]
GPT-4o	static	85	270	0.315	[0.262, 0.373]
	blind replay	44	270	0.163	[0.124, 0.212]
	rules-only	84	270	0.311	[0.259, 0.369]
Kimi K2	static	35	270	0.130	[0.095, 0.175]
	blind replay	52	270	0.193	[0.150, 0.244]
	rules-only	57	270	0.211	[0.167, 0.264]
DeepSeek V3	static	39	270	0.144	[0.108, 0.191]
	blind replay	28	270	0.104	[0.073, 0.146]
	rules-only	40	270	0.148	[0.111, 0.195]
Qwen3-32B	static	117	270	0.432	[0.376, 0.493]
	blind replay	63	270	0.233	[0.187, 0.287]
	rules-only	142	270	0.526	[0.466, 0.585]

TABLE 2. Paired McNemar's tests comparing static versus each adaptive condition, paired by (scenario, episode, drift-family) triple ($n = 270$ paired trials per comparison). n_{10}: pairs where static succeeded and adaptive failed; n_{01}: pairs where adaptive succeeded and static failed. χ^2: continuity-corrected Edwards statistic. † Static significantly outperforms adaptive ($p < 0.05$). ‡ Adaptive significantly outperforms static ($p < 0.05$).

Model	Comparison	n_{10}	n_{01}	χ^2	p
GPT-4o mini	static vs blind replay	59	13	28.12	<0.001†
	static vs rules-only	22	10	3.78	0.052
GPT-4o	static vs blind replay	48	7	29.09	<0.001†
	static vs rules-only	14	13	0.00	1.000
Kimi K2	static vs blind replay	4	21	10.24	0.001‡
	static vs rules-only	3	25	15.75	<0.001‡
DeepSeek V3	static vs blind replay	12	1	7.69	0.006†
	static vs rules-only	11	12	0.00	1.000
Qwen3-32B	static vs blind replay	69	15	33.44	<0.001†
	static vs rules-only	27	52	7.29	0.007‡

TABLE 3. Normalization-ablation results: static condition with and without the drift-inversion layer. The "with normalization" rates match those in Table 1 (static column); the "without normalization" rates are from a separate single-condition run not subject to the same data correction applied to the main results—success-rate magnitudes should be interpreted cautiously, though the early-termination asymmetry (GPT-4o mini $n = 236$ vs. $n = 270$ for all other models) is a structural observation independent of rate values. †$n = 236$: 30 toolset-drift episodes terminated early (unmapped tool names). See Section V for the schema-obedience interpretation.

Model	n (norm)	Rate (norm)	n (no norm)	Rate (no norm)
GPT-4o mini	270	0.396	236†	0.322
GPT-4o	270	0.315	270	0.204
Qwen3-32B	270	0.432	270	0.211
Kimi K2	270	0.130	270	0.130
DeepSeek V3	270	0.144	270	0.104

"without normalization" rates are from a separate run not subject to the same data correction as the main results (see Table 3 caption), the magnitude of per-drift-type effects without normalization should be interpreted cautiously; the early-termination asymmetry between GPT-4o mini and the other four models is a structural observation independent of the rate values.

Result 3 (Schema-Obedience as Moderator of Normalization Benefit). The early-termination asymmetry above constitutes a named secondary result: schema-obedience determines whether the drift-inversion normalization layer is load-bearing. GPT-4o mini faithfully invokes the injected drifted tool names; without the normalization layer, the environment rejects these calls, causing early terminations. The other four models invoke canonical tool names from training-time knowledge and produce no unmapped-tool-name errors with or without normalization. Schema-obedience thus moderates the normalization benefit: it is most directly useful for schema-obedient models, while models that fall back on training-time tool knowledge avoid unmapped-tool-name errors but are also unresponsive to intentional schema updates, representing a distinct deployment risk. A secondary diagnostic from the episode records is the average matched-steps field: premature-termination failures average 6.9 matched steps before `agent_stop` (i.e., failures typically occur mid-episode, not at the first action), while step-limit-exhaustion failures reach the maximum budget (12 steps), confirming that both categories reflect a convergence failure rather than an argument-formation failure.

V. DISCUSSION

The main finding is diagnostic rather than algorithmically novel: once evaluation-time reference trajectories are removed and success is defined by real environment completion, the reference-guided replay advantage does not transfer to the reference-free regime. Static baselines span a wide range across the five models—from 0.130 (Kimi K2) to 0.432 (Qwen3-32B). Blind replay consistently degrades performance relative to static for four of five models; Kimi K2 is the exception, for which blind replay is significantly better than static. Rules-only shows model-dependent effects, significantly improving over static for Kimi K2 and Qwen3-32B while showing no significant difference for GPT-4o and DeepSeek V3. These outcomes establish that reference-guided replay should be interpreted as an upper-bound benchmark condition, not as evidence that reference-free adaptation under drift is solved within the evaluated controller set.

The cross-model pattern is consistent with the hypothesis that the binding bottleneck has shifted from "can the system remember a previous action template?" to "can the system reconstruct the right hidden state and choose the correct branch among multiple plausible orders, products, and variants?" Blind replay—the strongest memory signal available in these experiments—did not improve outcomes for four of five models; for Kimi K2, where blind replay was effective, the improvement was uniform across all three drift families rather than drift-type specific, suggesting model-level retrieval coherence rather than drift-specific reasoning. This hypothesis is consistent with the observed data but not confirmed mechanistically; the failure-mode evidence described below provides supporting but indirect evidence.

A mechanistic account of why blind replay degrades performance is as follows. The blind replay controller retrieves a step-0 hint derived from a training episode, which encodes a specific entity identifier—an order ID, product variant, or customer record—from the training task. Under drift, the model incorporates this entity into its tool call not because it misunderstands the task structure, but because the retrieved hint presents a plausible-looking action cue that the model cannot distinguish from the current task's target without access to the full hidden state. The result is that the agent operates on the wrong branch (wrong order, wrong product) and either terminates with an incorrect database state (premature termination) or exhausts its step budget searching for a consistent sequence (step-limit exhaustion). This mechanism is specific to *replay-as-implemented-in-this-design*—a single step-0 hint retrieved by task type and drift family, without trajectory-level context or intent-cue indexing—and does not constitute a claim about replay as a category of adaptation strategy. More sophisticated retrieval mechanisms, such as STITCH-like intent-cue indexing with latent goal and salient entity encoding [21], might avoid the branch-confusion failure while still employing a replay-based design; the negative result here is therefore scoped to shallow, single-step hint retrieval under the current protocol.

Disaggregating the 270 eval trials by drift family reveals modest per-drift variation. For GPT-4o mini static, schema-drift yielded 37/90 (0.411), policy-drift 39/90 (0.433), and toolset-drift 31/90 (0.344)—a spread of 0.089, with toolset drift showing the largest disadvantage. GPT-4o and Qwen3-32B show near-uniform performance across drift types (GPT-4o: schema 0.322 / policy 0.300 / toolset 0.322; Qwen3-32B: schema 0.444 / policy 0.433 / toolset 0.422), indicating that drift type is not the primary driver of task difficulty for these models. Kimi K2 similarly shows near-uniform static performance across drift families (schema 13/90 = 0.144, policy 11/90 = 0.122, toolset 11/90 = 0.122; spread

of 0.022), consistent with the cross-drift uniformity of its adaptive-condition gains. DeepSeek V3 is at floor uniformly across all three drift families (0.144 for each), confirming drift-type independence. Under adaptive conditions, neither blind replay nor rules-only produces drift-type-specific improvements for any model, consistent with a failure mode that operates across drift operators.

The normalization-ablation condition (Table 3; see caption for the data-quality note on ''without normalization'' rates) shows that the drift-inversion layer contributes materially to most models. Removing normalization reduces GPT-4o mini from 0.396 to 0.322 overall and produces $n = 236$ rather than 270 trials: 30 toolset-drift episodes terminated early due to upstream errors on unmapped tool names. This early-termination asymmetry is absent for the other four models ($n = 270$ in all cases), revealing a secondary finding: *schema-obedience mediates normalization benefit*. GPT-4o mini faithfully calls the tool name as injected (e.g., `terminate_order`); without the drift-inversion layer, the environment rejects these calls. The other four models invoke canonical tool names from training knowledge—no unmapped-tool-name errors occur. A drift-inversion normalization layer is therefore most directly useful when deployed models are schema-obedient; models that fall back on training-time tool knowledge avoid unmapped-tool-name errors but are also unresponsive to intentional schema updates, representing a distinct deployment risk.

Examining episode-level failure records provides behavioral support for the branch-selection hypothesis. Across all conditions, invalid-tool-call rates are effectively zero, indicating that tool-format errors do not drive task failure. Behavioral failures concentrate in two categories: premature termination (`agent_stop` with `db_match=False`, where the model signals task completion but the database state does not match the expected outcome) and step-limit exhaustion (`max_steps`, where the model makes valid tool calls but cannot converge on the correct action sequence within budget). In the GPT-4o mini static condition, 136 of 163 failures (83%) are premature termination and 27 (17%) are step-limit exhaustion. Under blind replay for the same model, this distribution shifts markedly: 92 of 209 failures (44%) are premature termination and 117 (56%) are step-limit exhaustion. A similar shift appears for Qwen3-32B—97% premature termination in static, 51% under blind replay (step-limit rising to 48%). The replay hint causes the model to search longer but does not provide the hidden-state disambiguation needed to select the correct branch; step-limit exhaustion therefore replaces premature termination as the dominant failure mode under blind replay. Table 4 summarises the breakdown for the best-observed conditions; behavioral analysis is complete for eleven of fifteen model-condition combinations (all five models under static, and GPT-4o mini, GPT-4o, and Kimi K2 under each adaptive condition). DeepSeek V3 and Qwen3-32B under both adaptive conditions have partial records due to API budget-exhaustion episodes and are not fully tabulated.

TABLE 4. Failure-mode breakdown for illustrative conditions (eval split, deduplicated). Premature termination: `agent_stop` with `db_match=False`. Step-limit: `max_steps`. Percentages are of total failure count per row. Qwen3-32B blind replay ($n_{fail} = 191$ behavioral, excl. 16 budget-exhaustion + 3 unknown-tool failures) is included for contrast; the remaining adaptive-condition cells for DeepSeek V3 and Qwen3-32B have partial behavioral records.

Model	Condition	Failures	Premature	Step-Limit
GPT-4o mini	static	163	136 (83%)	27 (17%)
GPT-4o mini	blind replay	209	92 (44%)	117 (56%)
Qwen3-32B	static	153	148 (97%)	5 (3%)
Qwen3-32B	blind replay	191	98 (51%)	93 (49%)

A representative schema-drift case study illustrates the branch-selection mechanism directly. In the static condition, the agent receives the current task description and tool schema, with the drifted argument key (e.g., `order_ref`) transparently mapped back to `order_id` by the drift-inversion layer. The agent issues a single-step tool call to the correct order record and the environment returns `db_match=True`. In the blind replay condition, the same model additionally receives a memory cue derived from a training episode for the same task type; the cue references a specific order identifier from the training record. The agent incorporates this identifier into its tool call—selecting the training-set order rather than the current task's target order—and terminates having modified the wrong database record (`db_match=False`). The model had the correct tool format and action type, but the memory cue introduced a plausible yet incorrect identifier that the agent could not distinguish from the current task's target without access to the full hidden state.

The bijective normalization layer is an analytical isolation mechanism rather than a deployable adaptation strategy. By specifying the drift inversion in advance and applying it transparently at the environment boundary, the evaluation separates ''can the agent adapt its behavior once the interface change is known?'' from ''can the agent autonomously detect that an interface has changed?'' This decomposition isolates adaptation difficulty—branch selection and hidden-state reconstruction—independently of drift-detection overhead, making the evaluation tractable and reproducible. It deliberately defers the harder problem: in production deployments, agents must detect interface changes and infer new schemas without a pre-specified manifest. Studying agents that must both detect and repair schema changes without such a manifest is an important open problem, noted further in Section VI.

A low-cost alternative hypothesis for the model-dependent effects in rules-only conditions is that a drift-aware meta-prompt—a task-level prompt explicitly informing the model about expected interface changes—could achieve similar or better results without an online rule-injection infrastructure. Testing this hypothesis is an important future-work direction, particularly for the models where rules-only shows positive effects. More elaborate controller variants—multi-step memory cues, observation-based argument repair

(using tool-call rejection signals to infer schema mappings), and planning-augmented disambiguation (querying for entity identifiers before committing to write actions)—were not evaluated in this study. These variants are motivated by the current findings: if single-step blind replay fails primarily due to branch-selection ambiguity, multi-step retrieval and observation-based repair address distinct sub-problems and may show different patterns. Evaluating these variants is a prioritized direction for follow-up.

Two additional evaluation directions are highlighted by the current findings. First, within-episode error-driven adaptation—where the agent uses tool-call rejection signals or database-mismatch feedback to update its approach mid-episode—was not evaluated here. Error signals are available in the environment, and a credible future direction is whether error-driven within-episode adaptation can close the branch-selection gap without requiring cross-episode memory. Second, a documented-but-ambiguous drift condition—where the agent is provided a changelog indicating that interface drift occurred but without a complete inversion manifest—represents an intermediate setting between the bijective-inversion protocol studied here and full unsupervised drift detection. Studying agent performance under partial specification is a natural extension of the current evaluation framework.

The retail domain was selected because tau2-bench provides reproducible multi-step transaction tasks with observable database state, making success verification tractable. All conclusions in this paper are scoped to this domain and evaluation; external validity to other tool-use environments remains to be established. More structured domains—SQL-based enterprise query interfaces, medical record APIs, industrial control systems—are likely to exhibit distinct drift sensitivity patterns and adaptation challenges, and empirical evaluation across domains is needed to assess generalizability.

The hidden-state reconstruction problem maps naturally onto partially observable Markov decision process (POMDP) formulations of sequential decision-making [37]. In a POMDP framing, the agent maintains a belief state over latent environment variables (order identifiers, product variants, payment and address records) and must select actions that are robust to belief uncertainty. The reference-free controllers evaluated here lacked any explicit belief-state representation: they committed to write actions without first resolving ambiguity over which order or product branch was relevant, leading to the branch-selection failures that account for the majority of task failures. A credible adaptation gain will likely require explicit state reconstruction, better candidate-branch scoring, or environment-aware planners that can query for missing latent variables before committing to write actions—belief-tracking approaches such as entity stores, order-state graphs, or schema-aligned world models represent principled directions for closing the adaptation gap identified in this study. STITCH-style structured intent tuples [21] and entity-type indexes represent a concrete instantiation of this direction that could be adapted to the tau2 retail domain.

VI. LIMITATIONS

Several limitations constrain interpretation. All conclusions in this paper are explicitly scoped to the three evaluated drift types (schema key rename, tool-name remap, policy step prepend), the 30-scenario retail-domain evaluation set, and the three reference-free controller conditions (static, blind replay, rules-only). Findings may not generalize to other drift types, domains, or controllers without additional experimental validation.

First, the three evaluated drift types represent a narrow and controlled slice of real-world interface drift. More complex forms—type changes and argument removal (schema drift), multi-step policy reordering (policy drift), and partial deprecation with fallback (toolset drift)—raise distinct evaluation challenges and are identified as priority extensions. Second, all evaluated tasks are domain-specific to the tau2 retail environment; external validity to other tool ecosystems is not established.

Third, the reference-free evaluation used 30 eval scenarios and 3 episodes per condition, yielding 270 trials per cell. Individual pairwise comparisons at small effect sizes remain statistically underpowered, as noted in Section IV; the inferential claims rest on the cross-model and cross-condition pattern rather than any specific pairwise difference. Fourth, the controller comparison is limited to three conditions; multi-step memory cues, observation-based argument repair, and planning-augmented variants were not evaluated. The evaluation therefore cannot confirm whether more elaborate controllers would show a different pattern. Fifth, the upper-bound and reference-free task-success evaluators are incommensurable by design: the former is exact-trajectory imitation, the latter is end-state task completion; the upper bound should not be read as a directly comparable deployable condition.

Sixth, the bijective normalization layer defers the drift-detection subproblem: in production deployments, agents must autonomously detect interface changes and infer new schemas without a pre-specified inversion manifest. The current protocol isolates adaptation difficulty under known-drift conditions; studying agents that must both detect and repair schema changes without a manifest is an important open problem discussed in Section V. Seventh, token-level cost and latency were not instrumented; precise per-episode token counts and wall-clock latency were not logged, and future evaluations should instrument these metrics to quantify the overhead of more elaborate controllers relative to static prompting.

Eighth, all five models were accessed via OpenRouter during a defined experiment window (2026-03-10) using the identifiers recorded in Section VIII. OpenRouter model providers may silently update underlying weights without changing the version identifier string; exact reproduction at a later date is therefore not guaranteed despite the detailed episode-level archive. Ninth, the evaluated model set does not include Anthropic frontier models (Claude Opus, Claude Sonnet), which are widely used as agent-task benchmarks. Their omission is a genuine gap; including them would

require a new experiment run with the same protocol, and they are identified as a priority for follow-up evaluation.

Tenth, within-episode error-driven adaptation—where the agent uses tool-call rejection signals or environment error feedback to revise behaviour during execution—was not evaluated; it represents a distinct and prioritized future direction, as discussed in Section V. Finally, a formal percentage-of-valid-steps progress metric was not implemented in this evaluation; the `matched_steps` field in the episode records serves as a partial proxy (see Result 3 in Section IV), but defining a principled progress metric independently of any reference trajectory is a non-trivial open problem for reference-free evaluation settings.

VII. CONCLUSION

This study contributes a controlled-drift benchmark and two-regime evaluation methodology for tool-using LLM agents, separating a reference-guided upper-bound condition from a reference-free adaptation regime across three mechanically verifiable drift types. The primary contribution is methodological and diagnostic rather than algorithmic: the evaluation framework exposes that reference-guided replay advantage does not transfer to reference-free task success within the evaluated controller set. Within the 30-scenario retail-domain evaluation and three drift types, blind replay consistently degrades task success relative to static for four of five models, while rules-only shows model-dependent effects—significantly improving over static for Kimi K2 and Qwen3-32B and showing no significant difference for the remaining three models. A normalization-ablation condition further shows that the drift-inversion layer is load-bearing for toolset recovery, with its benefit mediated by schema-obedience. Within this scope, the primary unsolved problem is not access to action cues but hidden-state reconstruction and branch selection across multiple candidate orders and products. Future work should prioritize environment-aware state retrieval, error-driven within-episode adaptation, and broader reference-free benchmarks across domains and drift types, rather than treating replay alone as sufficient evidence of solved adaptation.

VIII. REPRODUCIBILITY

The reproducibility package for this study includes deterministic split manifests, episode-level evaluation records for all reported model-condition cells, frozen aggregate result tables, and minimal rerun configuration files. These materials are curated as a single archival artifact for independent verification and long-term access. Model API calls were routed through OpenRouter; the model version identifiers active at the time of the experiments (2026-03-10) were `openai/gpt-4o-mini` (GPT-4o mini), `openai/gpt-4o` (GPT-4o), `moonshotai/kimi-k2` (Kimi K2), `deepseek/deepseek-chat` (DeepSeek V3), and `qwen/qwen3-32b` (Qwen3-32B). OpenRouter does not expose immutable model content hashes; to support long-term reproducibility, the reproducibility package includes timestamped API call logs and the full episode records from which all reported results were derived, allowing independent verification of output consistency. The reproducibility package is archived on Zenodo [38]. The package also includes the analysis scripts used to produce all reported statistics. Code and data will be released under the MIT license concurrent with publication.

DATA AVAILABILITY AND LICENSING

The evaluated scenarios are derived from tau2-bench and adapted into a replay-compatible retail subset for controlled drift evaluation. The upstream tau2-bench source is distributed under the MIT license [35]. This manuscript reports results generated via OpenRouter endpoints for three model families [36]. The study dataset slice, split manifests, result records, provenance notes, and integrity metadata are included in the same archival reproducibility package; the dataset and analysis scripts are archived on Zenodo [38].

ACKNOWLEDGMENT

During the preparation of this manuscript, the authors used generative AI tools for specific tasks as follows: Claude and ChatGPT were used for manuscript drafting and editing assistance; Perplexity, Consensus, and SciSpace were used for literature search and synthesis; Codex was used for experiment scaffolding and analysis script development. All generated output was reviewed and verified by them, who take full responsibility for the content of this publication.

REFERENCES

[1] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi, and H. Wang, "Continual learning of large language models: A comprehensive survey," *ACM Comput. Surv.*, vol. 58, no. 5, pp. 1–42, Apr. 2026.

[2] J. Zheng, C. Shi, X. Cai, Q. Li, D. Zhang, C. Li, D. Yu, and Q. Ma, "Lifelong learning of large language model based agents: A roadmap," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Jan. 5, 2026, doi: 10.1109/TPAMI.2025.3650546.

[3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers Comput. Sci.*, vol. 18, no. 6, Dec. 2024, Art. no. 186345.

[4] S. Vatsal, H. Dubey, and A. Singh, "Agentic AI in healthcare and medicine: A seven-dimensional taxonomy for empirical evaluation of LLM-based agents," *IEEE Access*, vol. 14, pp. 4840–4863, 2026.

[5] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

[6] M. Riemer, I. Cases, R. Ajemian, M. Liu, I. Rish, Y. Tu, and G. Tesauro, "Learning to learn without forgetting by maximizing transfer and minimizing interference," in *Proc. Int. Conf. Learn. Represent.*, 2018.

[7] G. Merlin, V. Lomonaco, A. Cossu, A. Carta, and D. Bacciu, "Practical recommendations for replay-based continual learning methods," in *Proc. Int. Conf. Image Anal. Process.*, 2022, pp. 548–559.

[8] R. Aljundi, L. Caccia, E. Belilovsky, M. Caccia, M. Lin, L. Charlin, and T. Tuytelaars, "Online continual learning with maximally interfered retrieval," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 11849–11860.

[9] J. S. Smith, L. Valkov, S. Halbe, V. Gutta, R. Feris, Z. Kira, and L. Karlinsky, "Adaptive memory replay for continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 3605–3615.

[10] Y. Liu, C. Si, K. Narasimhan, and S. Yao, "Contextual experience replay for self-improvement of language agents," 2025, *arXiv:2506.06698*.

[11] E. Rabinovich and A. Anaby-Tavor, "On the robustness of agentic function calling," 2025, *arXiv:2504.00914*.

[12] W. Xiong, K. Wang, Y. Song, H. Liu, S. Zhou, W. Peng, and S. Li, "More vulnerable than you think: On the stability of tool-integrated LLM agents," 2025, *arXiv:2506.21967*.
[13] P. Yu, W. Liu, Y. Yang, J. Li, Z. Zhang, X. Feng, and F. Zhang, "Benchmarking LLM tool-use in the wild," in *Proc. Int. Conf. Learn. Represent.*, Mar. 2026. Accessed: Mar. 2026. [Online]. Available: https://openreview.net/forum?id=yz7fL5vfpn
[14] Z. Guo, S. Cheng, H. Wang, S. Liang, Y. Qin, P. Li, Z. Liu, M. Sun, and Y. Liu, "Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models," in *Proc. Findings Assoc. Comput. Linguistics*, 2024, pp. 1546–1568.
[15] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena: A realistic web environment for building autonomous agents," in *Proc. ICLR*, 2024.
[16] R. Cao, J. Chen, Z. Cheng, T. Hua, F. Lei, X. Li, Y. Liu, S. Savarese, D. Shin, T. Xie, C. Xiong, Y. Xu, T. Yu, D. Zhang, S. Zhao, V. Zhong, and S. Zhou, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," in *Proc. Adv. Neural Inf. Process. Syst. 37*, 2024, pp. 52040–52094.
[17] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an end-to-end web agent with large multimodal models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 6864–6890.
[18] X. Liu et al., "AgentBench: Evaluating LLMs as agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
[19] H. M. Pysklo, A. Zhuravel, and P. D. Watson, "Agent-Diff: Benchmarking LLM agents on enterprise API tasks via code execution with state-diff-based evaluation," 2026, *arXiv:2602.11224*.
[20] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "MCP-bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," 2025, *arXiv:2508.20453*.
[21] R. Yang, Y. Jiang, Y. Jiang, P. Kargupta, Y. Zhang, and J. Han, "Grounding agent memory in contextual intent," 2026, *arXiv:2601.10702*.
[22] H. Mishra, "RDumb++: Drift-aware continual test-time adaptation," 2026, *arXiv:2601.15544*.
[23] L. Caccia, R. Aljundi, N. Asadi, T. Tuytelaars, J. Pineau, and E. Belilovsky, "New insights on reducing abrupt representation change in online continual learning," in *Proc. Int. Conf. Learn. Represent.*, 2021.
[24] X. Han, H. Lu, H. Wang, and D. Gong, "Balancing plasticity and stability via dual-branch learning in online continual learning," in *Proc. Companion ACM Web Conf.*, May 2025, pp. 1014–1018, doi: 10.1145/3701716.3715590.
[25] A. Cano and B. Krawczyk, "ROSE: Robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams," *Mach. Learn.*, vol. 111, no. 7, pp. 2561–2599, Jul. 2022, doi: 10.1007/s10994-022-06168-x.
[26] A. Ashrafee, J. Kozal, M. Woźniak, and B. Krawczyk, "Holistic continual learning under concept drift with adaptive memory realignment," in *Proc. Trans. Mach. Learn. Res.*, Feb. 2026. [Online]. Available: https://openreview.net/forum?id=1drDlt0CLM
[27] S. Hao, Y. Gu, H. Ma, J. Hong, Z. Wang, D. Wang, and Z. Hu, "Reasoning with language model is planning with world model," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 8154–8173.
[28] J. Gonzalez, S. Patil, X. Wang, and T. Zhang, "Gorilla: Large language model connected with massive APIs," in *Proc. Adv. Neural Inf. Process. Syst. 37*, 2024, pp. 126544–126565.
[29] Z. Qi, X. Liu, I. L. Iong, H. Lai, X. Sun, Z. Wenyi, Y. Yu, X. Yang, J. Sun, S. Yao, T.-H. Zhang, W. Xu, T. Jie, and Y. Dong, "WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, 2024.
[30] R. Aljundi, M. Lin, B. Goujaud, and Y. Bengio, "Gradient based sample selection for online continual learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
[31] P. W. Koh et al., "WILDS: A benchmark of in-the-wild distribution shifts," in *Proc. 38th Int. Conf. Mach. Learn.*, 2020, pp. 5637–5664.
[32] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
[33] F. Cassano, A. Gopinath, K. Narasimhan, N. Shinn, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. 36*, 2023, pp. 8634–8652.
[34] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," in *Proc. Trans. Mach. Learn. Res.*, Mar. 2023. [Online]. Available: https://openreview.net/forum?id=ehfRiF0R3a
[35] Sierra Research. (2026). *TAU2-Bench*. Accessed: Mar. 2026. [Online]. Available: https://github.com/sierra-research/tau2-bench
[36] OpenRouter. (2026). *Openrouter Api Documentation*. Accessed: Mar. 2026. [Online]. Available: https://openrouter.ai/docs/api-reference/overview
[37] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artif. Intell.*, vol. 101, nos. 1–2, pp. 99–134, May 1998.
[38] M. H. Assidiqi, D. Alghazzawi, and S. Alarifi, "Benchmarking LLM agent robustness under tool-environment drift: Reproducibility package," Zenodo, King Abdulaziz Univ., Jeddah, Saudi Arabia, Tech. Rep., 2026, doi: 10.5281/zenodo.18995498.



MOHAMMAD HASBI ASSIDIQI received the B.S. degree in informatics and the M.S. degree in telematics from the Institut Teknologi Sepuluh Nopember (ITS), Surabaya, Indonesia, in 2008 and 2011, respectively. He is currently pursuing the Ph.D. degree in computer information systems with King Abdulaziz University, Jeddah, Saudi Arabia. He is an Assistant Professor with the Department of Creative Multimedia Technology, Politeknik Elektronika Negeri Surabaya (PENS), Indonesia. His research interests include large language model agents, online adaptation, machine learning, and applied AI systems.



DANIYAL ALGHAZZAWI (Senior Member, IEEE) received the Ph.D. degree in computer science from The University of Kansas, Lawrence, USA, in 2007. He is currently a Professor in cybersecurity with King Abdulaziz University, Jeddah, Saudi Arabia, and the Head of the Information Security Research Group. He has authored more than 175 scholarly articles, holds five patents, and actively contributes to seminars, publications as a reviewer/editor, and consults on information security policies and accreditations. His areas of specialization encompass smart e-learning, cybersecurity, and digital transformation.

SUAAD ALARIFI (Member, IEEE) received the Ph.D. degree in information security from Royal Holloway, University of London, Egham, U.K., in 2015. She is currently an Assistant Professor with the Department of Information Systems, King Abdulaziz University, Jeddah, Saudi Arabia. She has authored several scholarly articles in international journals and conferences. Her recent work focuses on developing cybersecurity risk assessment tools for non-governmental organizations (NGOs) and enhancing security frameworks for small and medium-sized enterprises (SMEs). She actively contributes to the academic community as a reviewer and a researcher, exploring the intersection of artificial intelligence and web application security. Her research interests include cybersecurity governance, cloud computing security, risk assessment, and anomaly detection.

LI CHENG received the Ph.D. degree in computer science from The University of Kansas, Lawrence, USA. He is currently affiliated with Xinjiang Technical Institute of Physics and Chemistry, Chinese Academy of Sciences, Ürümqi, China. He has been a Research Scientist for several research institutes more than 20 years, including Chinese Academy of Sciences, Biotechnology Software Institute, and The University of Kansas. His research interests include artificial intelligence, big data analytics, bioinformatics, and blockchain. He is the principal investigator (PI) and a Co-PI for a number of major research and development projects. He served on the expert committees for the National Natural Science Foundation of China, the Ministry of Science and Technology of the People's Republic of China, and other professional organizations.